

Technical Design Document (TDD)

Smart Insurance Claim Management System

1. Architecture Overview

The system follows Microservice Architecture using Spring Boot microservices communicating asynchronously through RabbitMQ.

2. Microservices

- customer-service
- policy-service
- claim-service
- notification-service
- config-server

3. Layered Architecture

Each microservice contains:

- Controller Layer
- Service Layer
- Repository Layer
- Entity Layer
- DTO Layer
- Mapper Layer
- Exception Layer
- Audit Layer
- Constants Layer

4. Database Design

H2 in-memory databases will be used for development and testing purposes.

5. REST APIs

CRUD APIs using GET, POST, PUT, DELETE methods for Customers, Policies, and Claims.

6. Exception Handling

Global exception handling implemented using `@RestControllerAdvice`. Custom runtime exceptions will handle validation and business rule failures.

7. Swagger/OpenAPI

Swagger documentation implemented using:

- `@Operation`
- `@Tag`
- `@ApiResponse`

8. Configuration Management

Spring Cloud Config Server stores configurations in GitHub repository.

9. Dynamic Refresh

GitHub webhooks trigger Spring Cloud Bus refresh using RabbitMQ.

10. Docker Deployment

All services are containerized using Docker images and managed using Docker Compose.

11. RabbitMQ Flow

Claim Service → RabbitMQ → Notification Service

12. Audit Design

Audit fields include `createdAt` and `updatedAt` using Spring Data JPA auditing.

13. Security & Validation

Input validations are implemented using Spring Validation annotations.

14. Technologies Used

- Java 17
- Spring Boot
- Spring Cloud
- RabbitMQ
- Docker
- H2 Database
- Lombok
- Swagger/OpenAPI